# Relatório de Viabilidade — Pipeline DevSecOps da API de Tarefas

Gustavo Costa Almeida · Ciência da Computação · Trilha DevSecOps SACI/UFLA 2026.2 · 12/10/2026 · Repositório:
**github.com/gustakakkoii/devsecops-trilha-gustavo-almeida** · Aplicação: **devsecops-trilha-gustavo-almeida.onrender.com**

## 1. ESCOPO E RESULTADO GERAL

A API (Node.js + Express, pasta `app/`) é analisada a cada push/PR por três gates no GitHub Actions: **Semgrep** (SAST), **Trivy** (SCA das dependências, misconfiguração do Dockerfile e scan da imagem) e **Gitleaks** (segredos no código atual e em todo o histórico do Git). O deploy no Render só é disparado quando **todos** passam.

| Gate | Critério de bloqueio | Antes da correção | Depois |
|---|---|---|---|
| **Semgrep** | achado ERROR ou WARNING | 6 achados (3 ERROR, 3 WARNING) | 0 |
| **Trivy · dependências** | HIGH/CRITICAL com correção | 5 HIGH | 0 |
| **Trivy · imagem** | HIGH/CRITICAL com correção | 23 CRITICAL · 577 HIGH (461 corrigíveis) | 0 CRITICAL · 0 HIGH |
| **Gitleaks** | qualquer segredo | 3 credenciais (código + histórico) | 0 (3 históricas rotacionadas) |

## 2. ACHADOS MAIS CRÍTICOS E REMEDIAÇÃO APLICADA

| Ferramenta | Achado | Impacto comprovado | Remediação |
|---|---|---|---|
| **Semgrep** | `eval()` com texto de `?expr=` em `tasks.js` (CWE-95) | Execução remota de código: `process.exit(0)` derrubou o servidor | Removido o `eval`; filtro passa a aceitar só os campos `done`, `owner` e `q` |
| **Semgrep** | `exec("ping -c 2 " + host)` em `network.js` (CWE-78) | Injeção de comando: `;id` retornou `uid=0(root)` | Sem shell: diagnóstico com `dns.lookup` nativo + validação do host (IP ou domínio) |
| **Semgrep** | `path.join` com nome vindo da URL em `files.js` (CWE-22) | Path traversal: leitura do `config.js` com as credenciais | `sanitizeFilename` (basename + allowlist) e verificação de que o caminho fica em `uploads/` |
| **Trivy** | Base `node:14` / Debian 10 (sem suporte): 21 CRITICAL e 545 HIGH no SO, ex.: CVE-2024-32002 (git), CVE-2023-38408 (openssh) | Superfície enorme (1,37 GB, shell, compiladores, npm) e CVEs públicas exploráveis | Build multi-stage; imagem final `distroless/nodejs24-debian13:nonroot`: 226 MB, sem shell e sem npm, usuário não-root |
| **Trivy** | `lodash 4.17.15` (CVE-2021-23337, CVE-2026-4800) e `jsonwebtoken 8.5.1` (CVE-2022-23539) | Injeção via template; validação fraca do tipo de chave em tokens | `lodash 4.18.1` e `jsonwebtoken 9.0.3`; `npm ci` instala exatamente o lockfile |
| **Gitleaks** | JWT secret e chaves AWS em texto puro em `config.js` (commit `7bf5c58`); exigiu regra própria, pois as padrão não detectaram | Qualquer pessoa com acesso ao repositório (ou a um clone antigo) obtém as credenciais | Removidas do código; `JWT_SECRET` no Environment do Render (secret manager) com valor novo (rotação); achados históricos registrados no `.gitleaksignore` |

## 3. RECOMENDAÇÕES

- **Atualização de versões:** manter o gate do Trivy e automatizar atualizações (Dependabot/Renovate); usar runtimes LTS (Node 24) e nunca imagens fora de suporte.
- **Imagens base mínimas/distroless:** build multi-stage, usuário não-root e rebuild periódico para receber patches da base. Menos pacotes significa menos CVEs e nenhum shell para um atacante.
- **Secret manager em vez de credenciais em texto puro:** segredos só em variáveis de ambiente/cofres (Render Environment, GitHub Secrets; em escala, Vault ou AWS Secrets Manager); `.env` no `.gitignore`; Gitleaks também como *pre-commit hook*. Segredo que vazou deve ser **rotacionado**, porque apagar do código não o remove do histórico.
- **Segurança da própria pipeline:** actions fixadas por SHA e ferramentas com checksum (lição do ataque de supply chain ao `trivy-action` em mar/2026), `GITHUB_TOKEN` somente leitura e branch protection exigindo os checks.
- **Próximos passos:** DAST (OWASP ZAP baseline contra o ambiente publicado), resultados SARIF no GitHub Code Scanning e publicar em registry a mesma imagem escaneada, para o Render executar exatamente o artefato aprovado.

> **Viabilidade:** a solução tem custo zero (GitHub Actions em repositório público + Render Free) e a execução completa, incluindo o deploy, levou cerca de 1 min 20 s. **Limitação conhecida do plano Free:** o serviço **hiberna após 15 minutos sem tráfego**, e a primeira requisição depois disso leva cerca de 1 minuto (cold start). Isso é esperado, não é falha. O sistema de arquivos também é efêmero, então as tarefas em memória se perdem a cada deploy ou reinício.